UNIVERSITATEA TEHNICĂ A MOLDOVEI

FACULTATEA CALCULATOARE, INFORMATICĂ ȘI MICROELECTRONICĂ

DEPARTAMENTUL INGINERIA SOFTWARE ȘI AUTOMATICĂ

Lucrare de laborator nr. 6

Tehnici de programare

Informații invizibile: forensics și steganografie

Realizat de : Bulat Madalina

st. gr. SI-262

Verificat de : Emil Cebotaros

asist. univ.

Chisinau 2026

1. **Scopul lucrării**

Scopul lucrării constă în studierea tehnicilor de analiză a datelor și fișierelor digitale, utilizând limbajul de programare Python. În cadrul lucrării au fost realizate două componente principale: Crearea unui decoder universal pentru identificarea și decodificarea diferitelor formate de codare. Analiza fișierelor prin tehnici de forensics și ascunderea, respectiv extragerea mesajelor prin steganografie LSB.

2. **Lucrarea A – Decoder universal**

A1. Conversia dintre text și octeți

În această etapă am studiat diferența dintre tipurile de date text și bytes.

Am utilizat metodele encode(), hex() și bytes.fromhex() pentru conversia textului în octeți și reprezentarea acestora în format hexadecimal.

A2. Identificarea stratului de codare

Am creat funcția ghici_strat(), care identifică tipul de codare al unui șir de caractere.

Funcția poate recunoaște:

URL – prin identificarea caracterului %.

HEX – prin identificarea caracterelor hexazecimale.

Base64 – prin verificarea alfabetului specific.

A3. Decodificarea stratului identificat

Am implementat funcția desfa(), care decodează șirurile în funcție de tipul identificat.

Au fost utilizate:

base64.b64decode() pentru Base64;

bytes.fromhex() pentru HEX;

unquote() pentru URL.

A4. Decodificarea straturilor multiple

Am realizat o buclă care decodează succesiv mai multe straturi până la obținerea unui rezultat lizibil.

Rezultatul obținut:

53595759

A5. Spargerea criptării XOR

Am analizat fișierul probe/xor.bin și am încercat toate cele 256 de chei posibile pentru o criptare XOR cu un singur octet.

Rezultatul obținut:

Cheia XOR: 42

Mesajul recuperat: FLAG{xor_a_fost_descoperit}

A6. Spargerea hash-ului prin dicționar

Am analizat hash-urile din fișierul probe/hashuri.txt, am identificat algoritmul după lungime și am comparat rezultatele hash-urilor generate pentru cuvintele din wordlist.

Rezultatul obținut:

Algoritm: MD5

Parola identificată: parola123

A7. Argparse, fișier JSON și testare

Am implementat posibilitatea de a transmite date programului prin intermediul liniei de comandă, folosind biblioteca argparse.

Rezultatul decodificării este salvat în fișierul fisa.json.

Am creat și testul automat teste/test_decode.py.

Rezultatul testării:

1 passed

Testul confirmă funcționarea decodificării unui șir Base64.

3. **Lucrarea B – Forensics și steganografie**

B1. Identificarea tipului real al fișierului

Am analizat primii octeți ai fișierului probe/foto.jpg, utilizând modul binar de citire.

Prin verificarea semnăturii fișierului am identificat tipul real al acestuia.

Rezultat: JPEG.

B2. Extragerea textului din fișierul binar

Am utilizat expresii regulate pentru identificarea secvențelor de caractere lizibile din fișierul probe/ascuns.png.

Am descoperit următoarele informații:

Numele fișierului ascuns: mesaj_ascuns.txt.

Mesajul identificat: Felicitari! Ai gasit fisierul ascuns.

Semnătura arhivei ZIP: PK.

B3. Extragerea metadatelor EXIF și GPS

Am analizat metadatele imaginii probe/foto.jpg utilizând biblioteca Pillow.

Am identificat următoarele informații:

Model: Forensics

Producător: Laborator

Latitudine: 47.0

Longitudine: 28.833333333333332

Coordonatele au fost convertite din grade, minute și secunde în grade zecimale.

B4. File carving – extragerea arhivei ascunse

Am identificat semnătura ZIP în fișierul probe/ascuns.png.

Am extras datele începând cu semnătura arhivei și am salvat rezultatul în fișierul gasit.zip.

După dezarhivare am identificat fișierul mesaj_ascuns.txt.

Mesajul recuperat:

Felicitări! Ai găsit fișierul ascuns.

B5. Ascunderea mesajului prin steganografie LSB

Am implementat metoda LSB (Least Significant Bit), care permite ascunderea informațiilor în ultimii biți ai canalelor de culoare ale pixelilor.

Am utilizat imaginea probe/foto.jpg ca imagine gazdă și am ascuns mesajul:

FLAG{ascuns_in_pixeli}

Imaginea rezultată a fost salvată în format PNG sub numele stego.png, pentru a evita pierderea biților modificați.

Rezultat: Mesajul a fost ascuns cu succes, iar imaginea stego.png a fost generată.

B6. Extragerea mesajului ascuns

Am citit ultimii biți ai canalelor RGB din imaginea stego.png.

Biții au fost grupați câte 8 pentru reconstruirea octeților, iar procesul de extragere s-a oprit la întâlnirea octetului nul.

Mesajul recuperat:

FLAG{ascuns_in_pixeli}

Rezultatul confirmă funcționarea metodei de ascundere și extragere a mesajului prin steganografie LSB.

Concluzie

În cadrul Laboratorului 6 am studiat metode de analiză criminalistică digitală și steganografie, utilizând limbajul Python. Am învățat să identific și să decodific date în diferite formate, să analizez mesaje codificate prin XOR și să verific hash-uri cu ajutorul unei liste de parole. De asemenea, am studiat structura fișierelor binare, extragerea textului și a metadatelor din imagini, identificarea fișierelor ascunse și tehnica LSB pentru ascunderea și recuperarea mesajelor. În urma acestor exerciții, mi-am dezvoltat abilitățile de programare și am înțeles importanța analizei datelor și a utilizării instrumentelor de securitate informatică în scopuri legale.

Bibliografie

https://visualstudio.microsoft.com/